

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. **Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.**

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ nel **caso pessimo** del seguente algoritmo MYSTERY1 quando
 - a. A è un Albero Binario di ricerca
 - b. A è un Albero AVL

Nota:

- COUNTNODES conta il numero di nodi nell'albero A utilizzando una visita in post-ordine
- SEARCH sfrutta la proprietà di ordinamento delle chiavi in un Albero Binario di Ricerca

Algorithm 1: MYSTERY1(A) $\rightarrow$ int

```

n = COUNTNODES(A)
k = 0
for i = 1, ..., n do
    x = MYSTERY2(i)
    if SEARCH(A, x) == true then
        k = k + 1
return k

function MYSTERY2(INT n)  $\rightarrow$  INT
if n ≤ 0 then
    return 1
else
    return MYSTERY2(n/2) + n3 + n2 + n + 1

```

Soluzione. Analizziamo prima il costo di MYSTERY2. Tale funzione richiama se stessa una volta su input $n/2$ e le altre operazioni nella chiamata hanno costo costante. L'equazione di ricorrenza di MYSTERY2

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ T'(n/2) + 1 & n > 1 \end{cases}$$

può essere risolta con il Master Theorem

$$\alpha = \log_2 2 = 1 = \beta \Rightarrow T'(n) = \Theta(n^\alpha \log n) = \Theta(\log n)$$

La complessità di MYSTERY1 nel caso pessimo dipende da:

1. **Costo della chiamata a COUNTNODES.** Indipendentemente dal fatto che A sia un Albero Binario di Ricerca o un Albero AVL, contare i nodi nell'albero ha costo lineare $\Theta(n)$ sul numero di nodi nell'albero.
2. **Costo del ciclo for.** Il ciclo viene eseguito n volte per i che va da 1 ad n (dove n è il numero di nodi nell'albero). In ogni iterazione esegue una chiamata a MYSTERY2 con input i ed una ricerca sull'albero A . Poiché il costo di MYSTERY2 è $\Theta(\log n)$, il suo contributo alla complessità è dato da:

$$\Theta(\log 1) + \dots + \Theta(\log n) = \Theta\left(\sum_{i=1}^n \log i\right) = \Theta\left(\log \prod_{i=1}^n i\right) = \Theta(\log n!) = \Theta(n \log n)$$

Il contributo della chiamata a SEARCH dipende dalla struttura dati sottostante e, in particolare, dall'altezza dell'albero. Abbiamo due casi:

- a. Quando A è un Albero Binario di Ricerca, l'altezza dell'albero è nel caso pessimo lineare sul numero di nodi (i.e. A è una lista). In questo caso, ogni chiamata a SEARCH ha costo pessimo $\Theta(n)$ e quindi n ricerche hanno costo pessimo complessivo $n \times \Theta(n) = \Theta(n^2)$. Aggiungendo il costo delle chiamate a MYSTERY2 otteniamo che il costo complessivo del ciclo for è:

$$\Theta(n \log n) + \Theta(n^2) = \Theta(n^2)$$

- b. Quando A è un Albero AVL, l'altezza dell'albero è sempre logaritmica rispetto al numero di nodi nell'albero. In questo caso, ogni chiamata a SEARCH ha costo pessimo $\Theta(\log n)$ e quindi n ricerche hanno costo pessimo complessivo $n \times \Theta(\log n) = \Theta(n \log n)$. Aggiungendo il costo delle chiamate a MYSTERY2 otteniamo che il costo complessivo del ciclo for è:

$$\Theta(n \log n) + \Theta(n \log n) = \Theta(n \log n)$$

Mettendo tutto insieme abbiamo che:

- a. Se A è un Albero Binario di Ricerca il costo di MYSTERY1 è:

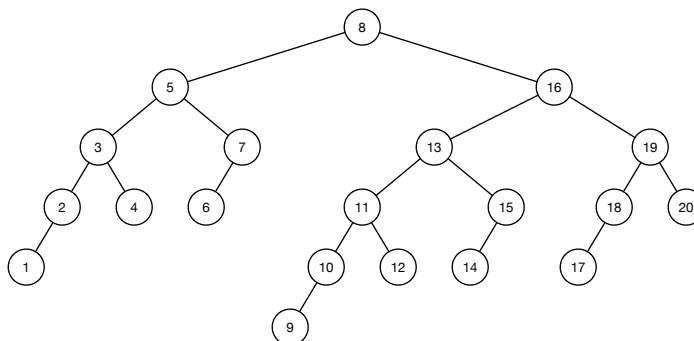
$$T(n) = \Theta(n) + \Theta(n^2) = \Theta(n^2)$$

- b. Se A è un Albero AVL il costo di MYSTERY1 è:

$$T(n) = \Theta(n) + \Theta(n \log n) = \Theta(n \log n)$$

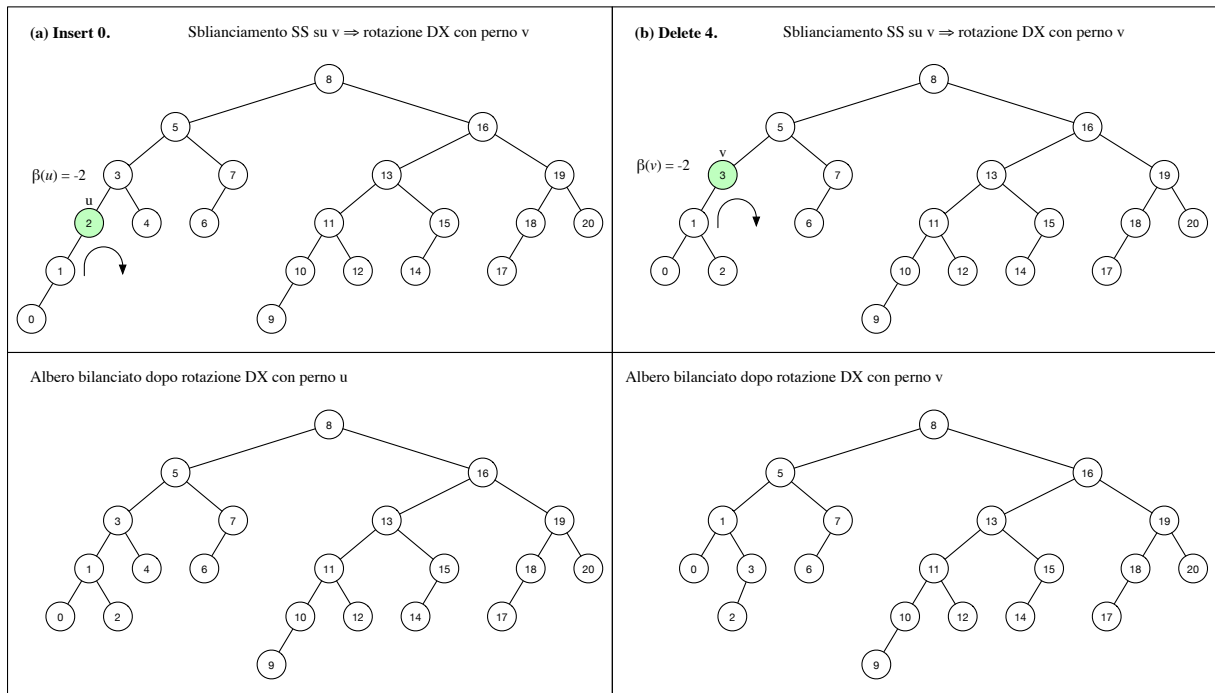
2. Dato il seguente albero AVL, effettuare le seguenti operazioni in ordine e mostrare lo stato dell'albero dopo ogni operazione

- a. Insert 0
- b. Delete 4



Indicare chiaramente quali operazioni sono eseguite in seguito ad inserimenti e/o rimozioni.

Soluzione.



3. Progettare un algoritmo che dati due array di numeri $A[1..n]$ e $B[1..m]$, entrambi ordinati in modo non decrescente, restituisce *true* se esiste un numero che appare sia in A sia in B (formalmente, esistono $i \in [1, n]$ e $j \in [1, m]$ tali che $A[i] = B[j]$), *false* altrimenti.

Soluzione. Essendo i due array ordinati, si può utilizzare una operazione simile all'operazione *merge* del mergesort. Si procede a leggere i due array in modo sequenziale iniziando dai loro numeri iniziali. Si confrontano i due valori letti, si restituisce immediatamente *true* se coincidono, altrimenti si passa al numero successivo nell'array che contiene il valore più piccolo fra i due letti. Si continua iterativamente in questo modo fino a che non si consumano tutti i numeri di uno dei due array. Se si arriva a tale situazione si restituisce *false*.

Tale algoritmo viene presentato in pseudocodice come Algoritmo 2. Nel caso pessimo è necessario leggere tutti i numeri nei due array, quindi il costo computazione risulta essere $O(n + m) = O(\max\{n, m\})$.

Algorithm 2: CERCAUGUALI(NUMBER[1..n] A , NUMBER[1..m] B) $\rightarrow$ BOOLEAN

```

INT  $i \leftarrow 1$ ,  $j \leftarrow 1$ 
while  $i \leq n$  AND  $j \leq m$  do
  if  $A[i] = B[j]$  then
    | return true
  else if  $A[i] < B[j]$  then
    |  $i \leftarrow i + 1$ 
  else
    |  $j \leftarrow j + 1$ 
return false

```

4. Un turista che vive in una città, deve raggiungere un'altra città, ma desidera attraversarne altre due durante il tragitto. Dobbiamo aiutarlo a capire la quantità minima di strada da percorrere. Più precisamente, bisogna progettare un algoritmo che dato un grafo orientato pesato $G = (V, E, w)$ con pesi non negativi (che rappresenta la mappa stradale) e quattro vertici distinti $v_1, v_2, v_3, v_4 \in V$ (che rappresentano rispettivamente la città di partenza, le due città intermedie e la città di arrivo) restituisce il costo del cammino di costo minimo che inizia da v_1 e arriva in v_4 , passando prima per v_2 e poi per v_3 .

Soluzione. Per risolvere il problema è possibile calcolare il cammino di costo minimo tra la prima e la seconda meta, dalla seconda alla terza, poi dalla terza alla destinazione definitiva, ed infine sommare questi costi. Per trovare il costo minimo di un tratto da percorrere si può utilizzare l'algoritmo di Dijkstra.

Tale algoritmo viene presentato in pseudocodice come Algoritmo 3. Vengono effettuate 3 chiamate all'algoritmo di Dijkstra, quindi il costo computazionale di tale algoritmo corrisponde con il costo di tale algoritmo, ovvero $O(m \log n)$, con $m = |E|$ e $n = |V|$.

Algorithm 3: ATTRAVERSAMENTO_CITTA($\text{GRAPH } G = (V, E, w)$, $\text{VERTEX } v_1, v_2, v_3, v_4$) $\rightarrow$ NUMBER

```

/* calcola e restituisce la somma dei cammini minimi dei tratti del percorso previsto */
return DIJKSTRA( $G, v_1, v_2$ )+DIJKSTRA( $G, v_2, v_3$ )+DIJKSTRA( $G, v_3, v_4$ )

/* calcola il costo del cammino minino dal vertice  $s$  al vertice  $d$  */
FUNCTION DIJKSTRA ( $\text{GRAPH } G = (V, E, w)$ ,  $\text{VERTEX } s, d$ )  $\rightarrow$  NUMBER
 $n \leftarrow G.\text{numNodi}()$ 
NUMBER  $D[1..n]$ 
for  $i \leftarrow 1$  to  $n$  do
     $D[i] \leftarrow \infty$ 
 $D[s] \leftarrow 0$ 
MINPRIORITYQUEUE[INT, NUMBER]  $Q \leftarrow \text{new MINPRIORITYQUEUE[INT, NUMBER]}()$ 
 $Q.\text{insert}(s, D[s])$ 
while not  $Q.\text{isEmpty}()$  do
     $u \leftarrow Q.\text{findMin}()$ 
    if  $u == d$  then
        /* trovato cammino minimo per  $d$  */
        return  $D[d]$ 
     $Q.\text{deleteMin}()$ 
    for  $v \in u.\text{adjacent}()$  do
        if  $D[v] = \infty$  then
            /* prima volta che si incontra  $v$  */
             $D[v] \leftarrow D[u] + w(u, v)$ 
             $Q.\text{insert}(v, D[v])$ 
        else if  $D[u] + w(u, v) < D[v]$  then
            /* scoperta di un cammino migliore per raggiungere  $v$  */
             $D[v] \leftarrow D[u] + w(u, v)$ 
             $Q.\text{decreaseKey}(v, D[v])$ 
return  $\infty$ 

```
